

SEGUNDO PARCIAL - FUNDAMENTOS DE INFORMÁTICA - RESPUESTAS TEÓRICAS

VECTORES Y MATRICES

¿Cuál es la diferencia entre variables locales y globales en c?

Locales: Declaradas dentro de funciones, solo accesibles desde ellas.

Globales: Declaradas fuera de funciones, accesibles desde todo el programa.

¿Qué es una función en el lenguaje de programación c y para qué se utiliza?

Una función es un bloque de código reutilizable que realiza una tarea específica. Permite dividir el programa en partes lógicas.

¿Qué es una librería en c y cómo se incluye en un programa?

Una librería es un conjunto de funciones y definiciones. Se incluye con #include <nombre.h>

Da un ejemplo de declaración y definición de una función que suma dos números:

```
int suma(int a, int b); //prototipo
int suma(int a, int b){
    return a + b;
}
```

1. ¿Qué es un vector y cómo se declara en un lenguaje de programación como C?

Un vector es una estructura de datos unidimensional que almacena un conjunto de elementos del mismo tipo. En C, se declara indicando el tipo de dato y el tamaño:

```
int numeros[10]; // vector de 10 enteros
```

2. ¿Cuál es la diferencia entre un vector y una matriz?

Un vector es un arreglo unidimensional, mientras que una matriz es un arreglo bidimensional (o de mayor dimensión) que permite almacenar datos en forma de tabla (filas y columnas).

3. ¿Cómo se accede a un elemento específico de una matriz bidimensional?

Usando el nombre de la matriz seguido de los índices entre corchetes:

```
matriz[fila][columna];
```

4. ¿Qué sucede si intentas acceder a un índice fuera del rango de un vector?

Se accede a una posición de memoria no asignada, lo que puede causar errores de ejecución o comportamientos inesperados (desbordamiento de memoria).

5. ¿Cómo se puede utilizar una función para trabajar con vectores en C?

Pasando el vector como parámetro a la función. Ejemplo:

```
void mostrar(int v[], int tam) {
    for (int i = 0; i < tam; i++)
        printf("%d ", v[i]);
}
```

ESTRUCTURAS (STRUCT)

1. ¿Qué ventajas ofrecen las estructuras respecto a los vectores y matrices?

Permiten agrupar diferentes tipos de datos bajo una sola entidad, representando objetos más complejos como personas, productos, etc.

SEGUNDO PARCIAL - FUNDAMENTOS DE INFORMÁTICA - RESPUESTAS TEÓRICAS

VECTORES Y MATRICES

2. Da un ejemplo de una estructura en C y explica sus componentes.

```
struct Persona {  
    char nombre[50];  
    int edad;  
    float altura;  
};
```

Contiene tres miembros: un string, un entero y un flotante.

3. **¿Para qué situaciones es más conveniente usar una estructura en vez de un vector?**

Cuando se necesita almacenar diferentes tipos de datos relacionados entre sí (por ejemplo, datos de un estudiante).

ESTRUCTURAS ANIDADAS (STRUCT)

1. **¿Qué es una estructura anidada y para qué se utiliza en programación?**

Es una estructura que contiene otra estructura como miembro. Se usa para representar relaciones jerárquicas o compuestas, como una persona que tiene una dirección.

2. **¿Cuáles son las ventajas de utilizar estructuras anidadas en lugar de estructuras simples con muchos miembros?**

Organiza mejor la información, mejora la legibilidad y facilita el mantenimiento del código.

3. **Explica con tus palabras cómo se pueden anidar estructuras usando structs en C.**

Se declara una estructura dentro de otra:

```
struct Direccion {  
    char calle[50];  
    int numero;  
};  
struct Persona {  
    char nombre[50];  
    struct Direccion direccion;  
};
```

4. **¿En qué casos conviene crear una nueva estructura dentro de otra?**

Cuando el conjunto de datos que se repite (como "direccion") puede ser reutilizado o tratado como unidad lógica.

5. **Da un ejemplo de dos estructuras (por ejemplo, Cliente y Empleado) que compartan miembros y cómo los anidarías.**

```
struct DatosContacto {  
    char telefono[20];  
    char email[50];  
};  
struct Cliente {  
    char nombre[30];  
    struct DatosContacto contacto;  
};  
struct Empleado {  
    char nombre[30];  
    struct DatosContacto contacto;  
};
```

SEGUNDO PARCIAL - FUNDAMENTOS DE INFORMÁTICA - RESPUESTAS TEÓRICAS

VECTORES Y MATRICES

6. ¿Cómo accederías al campo "Dirección" de un cliente usando structs anidados?

```
cliente.direccion.calle;
```

FUNCIONES

1. ¿Qué es una función en el lenguaje de programación C y para qué se utiliza?

Una función es un bloque de código reutilizable que realiza una tarea específica. Permite dividir el programa en partes lógicas.

2. ¿Cuáles son las ventajas de usar funciones en C?

Reutilización de código, modularidad, facilidad de mantenimiento y mejora en la legibilidad.

3. Explica la diferencia entre una función que retorna un valor y una que no retorna nada (void).

Una función con valor devuelve un resultado al programa. Una `void` solo ejecuta instrucciones sin devolver datos.

4. ¿Qué es un prototipo de función y por qué es importante?

Es la declaración previa de una función. Informa al compilador del nombre, tipo de retorno y parámetros. Ayuda a evitar errores de tipo y orden.

5. ¿Cuál es la diferencia entre pasar parámetros por valor y por referencia?

Por valor: se pasa una copia del dato. Por referencia (con punteros): se pasa la dirección de memoria, permitiendo modificar el valor original.

6. Da un ejemplo de declaración y definición de una función que suma dos números.

```
int suma(int a, int b); // prototipo
```

```
int suma(int a, int b) {
```

```
    return a + b;
```

```
}
```

7. ¿Por qué es importante la reutilización de funciones en un programa?

Evita repetir código, mejora la eficiencia en el desarrollo y reduce errores.

EXTRAS

1. ¿Cuál es la diferencia entre variables locales y globales en C?

Locales: declaradas dentro de funciones, solo accesibles desde ellas.

Globales: declaradas fuera de funciones, accesibles desde todo el programa.

2. ¿Qué es una librería en C y cómo se incluye en un programa?

Una librería es un conjunto de funciones y definiciones. Se incluye con `#include`

```
<nombre.h>.
```

3. ¿Qué es la sentencia switch y cuándo es preferible usarla en lugar de if-else?

`switch` permite comparar una variable contra varios valores. Es más legible y eficiente cuando hay muchas condiciones fijas.

4. Explica con un ejemplo práctico cuándo usar switch para crear un menú de opciones.

```
int opcion;
```

```
printf("1. Alta\n2. Baja\n3. Salir\n");
```

```
scanf("%d", &opcion);
```

```
switch(opcion) {
```

```
    case 1: printf("Alta\n"); break;
```

```
    case 2: printf("Baja\n"); break;
```

```
    case 3: printf("Salir\n"); break;
```

```
    default: printf("Opcion invalida\n");
```

```
}
```